FIT2109 Computer Science Workshop

Week 6: Virtual Machines, Containers & Package Management

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

The problem this week solves

"It works on my machine."

The most expensive sentence in software engineering.

- Different operating systems, library versions, and system tools produce different behaviour.
- A working laptop is not the same environment as a CI server, a teammate's laptop, or a production host.
- Reproducing bugs, on-boarding new contributors, and deploying reliably all depend on solving this problem.

Where this week fits

Shell $\rightarrow$ scripting $\rightarrow$ Git $\rightarrow$ **environments**

$\rightarrow$ remote machines $\rightarrow$ debugging $\rightarrow$ automation

Week 6 goal

Understand the tools that make software environments reproducible: package managers, containers, and virtual machines.

Learning goals

By the end of this workshop, you should be able to:

- Explain why the same software can behave differently on different machines.
- Create an isolated Python environment with `venv` and pin dependencies in `requirements.txt`.
- Install packages with `pip`, `npm`, and OS package managers such as `apt`.
- Explain what a lockfile is and why it matters.
- Use core Docker commands to run, inspect, and remove containers.
- Read a Dockerfile and explain what each instruction does.
- Contrast containers and virtual machines and state when each is appropriate.

Concept 1: Machines differ in more than operating system

Things that differ between machines

- Operating system and version
- System libraries (`libc`, `openssl`)
- Language runtime versions (Python 3.9 vs 3.12)
- Package versions
- Environment variables and configuration

Consequences

- Tests pass locally, fail in CI.
- A library upgrade on one machine breaks another.
- A deployed app crashes in ways that cannot be reproduced locally.
- Debugging requires guessing the environment.

Concept 2: pip installs; venv isolates

pip — Python's package installer

```
pip install requests       # install
pip install requests==2.31.0  # pin version
pip uninstall requests
pip list                   # installed
    packages
pip show requests          # details
```

Problem: global installs conflict

Project A needs `Django==3.2`. Project B needs `Django==4.2`. A global install can only hold one version.

venv — per-project isolation

```
python3 -m venv .venv
source .venv/bin/activate  # prompt: (.venv)
pip install requests
pip freeze > requirements.txt
deactivate  # PATH restored; .venv/ intact
```

requirements.txt — the lockfile

```
source .venv/bin/activate
pip install -r requirements.txt
```

Demo 4: venv and requirements.txt

Watch

Create a Python virtual environment, install declared dependencies, run a small program, then deactivate.

Note

venv isolates Python packages on the host. Docker isolates the runtime filesystem and process environment.

Handout Activity 1: Python package evidence

Now work on this activity in the handout

Read a small Python setup and decide what `pip`, `venv`, and `requirements.txt` each prove.

Group output

Complete the command/evidence table and explain why a global install is not a reproducible project setup.

Concept 3: npm records Node dependencies

Key commands

```
npm install express          # runtime
    dependency
npm install --save-dev jest # dev-only
npm list                     # installed
    packages
```

package.json — the manifest

Records: dependencies, devDependencies, and scripts (command shortcuts). Created with npm init.

package-lock.json

Auto-generated. Pins *exact* versions of every package, including transitive ones. Commit this file.

node_modules/

Never commit — can be hundreds of MB. Add to .gitignore and regenerate with npm install.

Demo 5: `npm init` and `npm install`

Watch

Create a tiny Node project, install a dependency, and compare `package.json` with the lockfile.

Note

Manifest files describe intent; lockfiles record the exact resolved dependency versions.

Concept 4: Lockfiles make versions repeatable

The pattern across ecosystems

Ecosystem	Manifest	Lockfile
Python	`requirements.txt`	`requirements.txt`*
Node.js	`package.json`	`package-lock.json`
Ruby	`Gemfile`	`Gemfile.lock`
Rust	`Cargo.toml`	`Cargo.lock`

* Python uses one file for both roles. Run `pip freeze > requirements.txt` to produce a pinned lockfile version.

Manifest vs lockfile

- **Manifest**: declares what you want, often with version ranges (`>=2.0`).
- **Lockfile**: pins exactly what was installed. Fully deterministic.

Rules

- Always commit the lockfile.
- Never hand-edit a lockfile.
- Add `node_modules/` and `.venv/` to `.gitignore`.

Concept 5: System packages install OS-level tools

apt — Debian, Ubuntu, WSL

```
sudo apt-get update
sudo apt-get install -y curl
sudo apt-get remove curl
apt-cache search json
```

Used in Dockerfiles: `RUN apt-get install ...`

brew — macOS

```
brew install node
brew upgrade node
brew list
```

yum / dnf — RHEL, Fedora, CentOS

```
sudo yum install curl
sudo dnf install curl    # modern alias
sudo yum remove curl
yum search json
```

Rule of thumb

apt on Ubuntu/Debian (including most Docker base images). yum/dnf on Red Hat-family systems. brew on macOS.

Handout Activity 2: dependency evidence across ecosystems

Now work on this activity in the handout

Compare Python, Node, Java, and OS-level package evidence. Decide which files should be committed and which generated folders should stay out of Git.

Group output

Complete the evidence table and the commit/do-not-commit list.

Concept 6: Containers isolate a process and filesystem

How a container works

A **container** is an isolated process with its own filesystem view, created using Linux kernel features (`namespaces` + `cgroups`).

The host kernel is shared. No second OS is booted.

Advantages over VMs

- Starts in milliseconds.
- Megabytes, not gigabytes, per image layer.
- Hundreds can run on one host.
- Identical image runs on any Linux-based host.

Limitations

- Containers share the host kernel — cannot run a Windows container on Linux natively.
- Weaker security boundary than a VM.
- On macOS/Windows, Docker runs a lightweight Linux VM internally.

Concept 7: Docker names the container workflow

Key terms

Image A read-only template. Think: a frozen filesystem + metadata.

Container A running instance of an image.

Registry A server that stores and distributes images (Docker Hub is the default).

Layer Each change to an image adds a layer; layers are cached and shared.

Analogy

Image	$\approx$ class definition
Container	$\approx$ instance of the class
Registry	$\approx$ package repository

Workflow

Pull image $\rightarrow$ `docker run` $\rightarrow$ process runs $\rightarrow$ container exits or is stopped

Demo 1: first steps with Docker

Watch

Pull an image, run short-lived containers, inspect images and container history, then clean up deliberately.

Note

Separate the image from the container: one is the reusable blueprint; the other is one run of that blueprint.

Demo 3: apt-get inside a container

Watch

Install an OS-level tool inside a disposable Ubuntu container and observe what disappears when the container is removed.

Note

`apt-get` changes a running environment; a Dockerfile records those changes so someone else can reproduce them.

Two Docker workflows: deployment vs development

Deployment-style — bake code into image

```
docker build -t myapp .
docker run --rm myapp
```

Source is baked in. Rebuild required after every code change. Use this for CI and production.

Development-style — bind-mount live source

```
docker run --rm -it \
  -v "$PWD":/app -w /app \
  python:3.12-slim \
  sh -lc "pip install -r requirements.txt \
          && python app.py"
```

Host directory mounted live — no rebuild needed on each edit.

Trade-off

Development-style is convenient but skips the Dockerfile. Run deployment-style before committing to confirm the image actually builds.

Concept 8: Dockerfiles describe image builds

Core instructions

`FROM`	Base image to start from.
`WORKDIR`	Working directory inside image.
`COPY`	Copy files from host into image.
`RUN`	Run a command during build.
`CMD`	Default command on container start.
`ENV`	Set an environment variable.
`EXPOSE`	Document which port the app uses.

Build and run

```
# In the directory with Dockerfile:
docker build -t myapp .

docker run --rm myapp
```

Layer caching tip

Copy `requirements.txt` *before* `COPY . .` so the `pip install` layer is cached unless dependencies change.

A Dockerfile for a Python app

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
ENV PYTHONDONTWRITEBYTECODE=1   # no .pyc files during install or at runtime
RUN pip install --no-cache-dir -r requirements.txt
# --no-cache-dir: skip pip's cache => smaller image
COPY . .
CMD ["python", "app.py"]
```

Why `requirements.txt` before `COPY . .`?

Docker caches each layer. If only source changes, the cached `pip install` layer is reused — much faster rebuild.

Handout Activity 3: Docker images and Dockerfiles

Now work on this activity in the handout

Annotate a Dockerfile, classify image/container statements, and explain when a bind mount changes the workflow.

Group output

Complete the image/container table, Dockerfile line explanations, and one development-vs-deployment comparison.

Demo 2: build and run a Python image

Watch

Build a small Python image, run it, inspect what is inside, then compare the build step with the run step.

Note

Watch the Dockerfile order. Dependency layers should change less often than source-code layers.

Concept 9: Multi-stage builds

Problem

Build tools are large but only needed at build time. Shipping them in the final image wastes space and increases attack surface.

Pattern: two `FROM` instructions

Stage 1 compiles. Stage 2 copies only the compiled output — no build toolchain in the final image.

```
# Stage 1: build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY . .
RUN mvn -q -DskipTests package

# Stage 2: runtime only
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /src/target/app.jar .
CMD ["java", "-jar", "app.jar"]
```

Result

Same app, smaller final image, fewer tools to secure.

Concept 10: Virtual machines isolate a whole OS

How a VM works

A **hypervisor** (VMware, VirtualBox, Hyper-V) emulates hardware so a guest OS boots on the host — each guest has its own kernel, memory, and disk image.

When to use a VM

- Testing on a different OS (e.g. Windows on Linux).
- Strong isolation required (own kernel, security boundary).
- Running legacy software tied to a specific OS state.

Cost of VMs

- Full OS boot (seconds to minutes).
- Gigabytes of disk per image.
- Memory reserved per guest.

Concept 11: Choose the environment tool by the problem

Tool	Scope	Isolation level
Language package manager	Libraries only	Per-language
System package manager	OS-level tools	Machine-wide or per-user
Container	Process + filesystem	OS-level (shared kernel)
Virtual machine	Full OS image	Complete (own kernel)

The right tool depends on the problem

A container is not a virtual machine. A virtual environment is not a container. Each solves a different slice of the environment problem.

Handout Activity 4: environment boundary and handoff

Now work on this activity in the handout

Choose between a package manager, container, VM, or combination for realistic setup problems, then turn one choice into a handoff workflow.

Group output

Complete the scenario table and write a short reproducible handoff plan.

Final checkpoint: Poll Everywhere

pollev.com/fit2109

Join today's checkpoint

Scan the QR code or go to pollev.com/fit2109.

Purpose

Use the Poll Everywhere questions to check which Week 6 ideas need one more example before we finish.

What should feel different after today?

You should now be able to

- Pull and run a Docker image, and explain what happened.
- Read a Dockerfile and explain each instruction.
- Install system packages with `apt-get` inside a container.
- Create a Python venv, install packages, and pin them.
- Initialise a Node project and understand `package.json`.
- Explain the difference between a manifest and a lockfile.

The idea to keep

Reproducibility = declared dependencies + locked versions + isolated runtime

Containers declare the runtime filesystem. Lockfiles pin library versions. Together they make "works on my machine" easier to reproduce and debug.

Q&A and next steps

Coming up

- **Applied class**: you will containerise a small application end-to-end.
- **Week 8**: Debugging — systematic bug hunting and evidence-based fixes.

Resources

- `seminar/guides/docker_install.md` — installation instructions you should have completed before today.
- `docs.docker.com/get-started` — official Docker tutorial.
- `python3 -m venv -help` and `npm help` — built-in docs.

Questions?